

Pointers in C

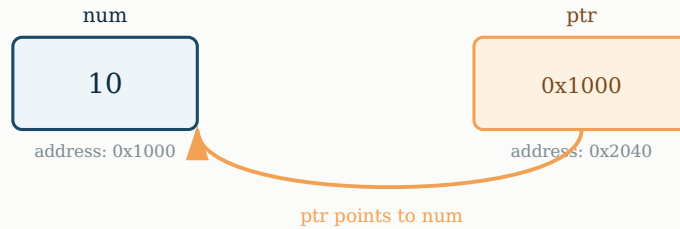
1. What is a Pointer?	03
2. Pointer Declaration Syntax	04
3. Address-of (&) and Dereference (*) Operators	04
4. NULL Pointers	05
5. Pointer Arithmetic	06
6. Pointers and Arrays	07
7. Pointers and Strings	08
8. Pointer to Pointer (Double Pointer)	09
9. Array of Pointers	10
10. Const Pointers — Three Variants	11
11. Common Pitfalls	12
12. Quick Summary Table	13

1 · What is a Pointer?

Every variable in a C program is stored somewhere in the computer's memory, and every memory location has a unique **address**. A **pointer** is a special variable that does not store ordinary data — it stores the **address** of another variable.

Simple idea: A normal variable is a box with a value inside. A pointer is a box that stores the *house number* of another box.

Diagram: Variable vs Pointer in Memory



Program 1.1

```
#include <stdio.h>

int main() {
    int num = 10;
    int *ptr = &num;    // ptr stores the address of num

    printf("Value of num: %d\n", num);
    printf("Address of num: %p\n", &num);
    printf("Value stored in ptr: %p\n", ptr);
    printf("Value pointed by ptr: %d\n", *ptr);

    return 0;
}
```

Sample Output

```
Value of num: 10
Address of num: 0x1000
Value stored in ptr: 0x1000
Value pointed by ptr: 10
```

Explanation, line by line:

- `&num` — the **address-of operator**. It gives the memory address where `num` is stored.
- `int *ptr` — declares `ptr` as a pointer that can hold the address of an `int` variable.
- `*ptr` — the **dereference operator**. It looks at the address inside `ptr` and fetches the value stored there.
- Notice that `&num` and the value printed for `ptr` are identical — because `ptr` is literally holding `num`'s address.

2 · Pointer Declaration Syntax

A pointer must be declared with the data type it will point to, so the compiler knows how to interpret the memory at that address.

Syntax

```
data_type *pointer_name;
```

Examples

```
int    *p;    // pointer to an int
char   *c;    // pointer to a char
float  *f;    // pointer to a float
double *d;    // pointer to a double
```

- The `*` symbol tells the compiler "this variable stores an address."
- The `data_type` tells the compiler what kind of value lives at that address — this matters for dereferencing correctly and for pointer arithmetic (Section 5).

3 · Address-of (&) and Dereference (*)

Changing a value through a pointer

Program 3.1

```
#include <stdio.h>

int main() {
    int x = 25;
    int *p = &x;

    *p = 50;    // changing the value through the pointer

    printf("x = %d\n", x);
    printf("*p = %d\n", *p);

    return 0;
}
```

Sample Output

```
x = 50
*p = 50
```

Diagram: Writing Through a Pointer



Explanation:

- `*p = 50;` does not change `p` — it changes the value stored at the address `p` is holding, which belongs to `x`.
- This is the core idea behind using pointers to modify data indirectly.

4 · NULL Pointers

A pointer that is not yet pointing to any valid memory location should be set to NULL. This avoids accidentally reading or writing to a random, unknown memory address.

Program 4.1

```
#include <stdio.h>

int main() {
    int *ptr = NULL;

    if (ptr == NULL) {
        printf("ptr is a null pointer, points to nothing.\n");
    }

    return 0;
}
```

Sample Output

```
ptr is a null pointer, points to nothing.
```

Rule of thumb: Always initialize a pointer — either give it a real address or set it to NULL. Dereferencing an uninitialized or NULL pointer (like `*ptr`) is undefined behavior and usually crashes the program.

5 · Pointer Arithmetic

Pointers can be moved forward or backward using addition and subtraction. Importantly, moving a pointer by 1 does not move it by 1 byte — it moves it by $1 \times \text{sizeof}(\text{data_type})$ bytes.

Program 5.1

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;    // array name decays into a pointer

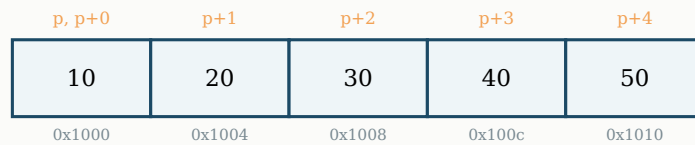
    for (int i = 0; i < 5; i++) {
        printf("Address: %p, Value: %d\n", (void*)(p + i), *(p + i));
    }

    return 0;
}
```

Sample Output

```
Address: 0x1000, Value: 10
Address: 0x1004, Value: 20
Address: 0x1008, Value: 30
Address: 0x100c, Value: 40
Address: 0x1010, Value: 50
```

Diagram: Pointer Walking Through an Array (int = 4 bytes)



Explanation:

- $p + i$ automatically jumps forward by $i \times \text{sizeof}(\text{int})$ bytes — the compiler handles the scaling.
- $*(p + i)$ gives the value at that position, exactly equal to $\text{arr}[i]$.
- This is why arrays and pointers are so closely linked in C.

6 · Pointers and Arrays

Program 6.1

```
#include <stdio.h>

int main() {
    int arr[4] = {1, 2, 3, 4};
    int *p = arr;

    printf("arr[2]    = %d\n", arr[2]);
    printf("(p+2)     = %d\n", *(p + 2));
    printf("p[2]      = %d\n", p[2]);

    return 0;
}
```

Sample Output

```
arr[2]    = 3
*(p+2)     = 3
p[2]      = 3
```

Four ways to read the same value: `arr[i] = *(arr + i) = *(p + i) = p[i]`

The array name itself acts like a constant pointer to its first element.

7 · Pointers and Strings

A C string is really just an array of characters ending in a special "null terminator" character `'\0'`. A char pointer can point to the first character and walk through the rest.

Program 7.1

```
#include <stdio.h>

int main() {
    char *str = "HELLO";
    char *p = str;

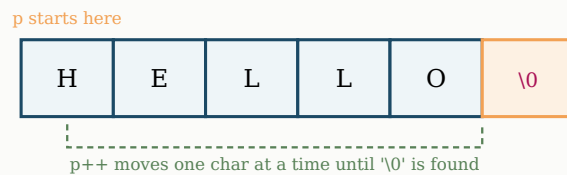
    while (*p != '\0') {
        printf("%c", *p);
        p++;
    }
    printf("\n");

    return 0;
}
```

Sample Output

```
HELLO
```

Diagram: Pointer Walking Through a String



Explanation:

- `str` points to the character `'H'`, the first character of the string.
- `*p != '\0'` keeps looping until the pointer reaches the hidden null terminator that marks the end of every C string.
- `p++` moves the pointer to the next character, one byte at a time (since `char` is 1 byte).

8 · Pointer to Pointer (Double Pointer)

A pointer can also store the address of **another pointer**. This is called a pointer to a pointer, written with two asterisks.

Program 8.1

```
#include <stdio.h>

int main() {
    int num = 100;
    int *p = &num;
    int **pp = &p;

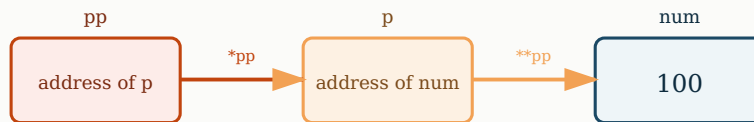
    printf("num   = %d\n", num);
    printf("**p   = %d\n", *p);
    printf("***pp  = %d\n", **pp);

    return 0;
}
```

Sample Output

```
num   = 100
*p    = 100
**pp  = 100
```

Diagram: Chain of Pointers



Explanation:

- `p` holds the address of `num`.
- `pp` holds the address of `p` — one level higher.
- `*pp` gives you `p`'s value (an address). Dereferencing again, `**pp`, follows that address to finally reach `num`'s value.

9 · Array of Pointers

Program 9.1

```
#include <stdio.h>

int main() {
    char *fruits[3] = {"Apple", "Banana", "Cherry"};

    for (int i = 0; i < 3; i++) {
        printf("%s\n", fruits[i]);
    }

    return 0;
}
```

Sample Output

```
Apple
Banana
Cherry
```

Explanation:

- fruits is an array where each slot is a char* pointing to a different string in memory.
- This is memory-efficient because each string can be a different length — unlike a fixed-width 2D character array.

10 · Const Pointers — Three Variants

Program 10.1

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    const int *p1 = &a;           // value fixed, pointer can move
    int *const p2 = &a;           // pointer fixed, value can change
    const int *const p3 = &a;     // both fixed

    p1 = &b;                       // OK: p1 can now point to b
    *p2 = 200;                     // OK: value at a can change

    printf("a = %d, b = %d\n", a, b);

    return 0;
}
```

Declaration	What is fixed	What can change
const int *p1	The value pointed to	Which address p1 holds
int *const p2	The address p2 holds	The value at that address
const int *const p3	Both	Neither

Explanation:

- Reading the declaration right-to-left from the pointer name helps: const int *p1 reads as "p1 is a pointer to a constant int."
- int *const p2 reads as "p2 is a constant pointer to an int."

11 · Common Pitfalls

Pitfall	What it means
Wild pointer	An uninitialized pointer used before it's given a real address
NULL dereference	Using <code>*ptr</code> when <code>ptr</code> is <code>NULL</code> — crashes the program
Out-of-bounds access	Using pointer arithmetic to read/write outside a valid array
Type mismatch	Assigning an address of one type to a pointer of a different type without a cast

Golden rule: Never dereference a pointer unless you are certain it currently holds a valid, meaningful address.

12 · Quick Summary Table

Concept	Syntax	Meaning
Address-of	<code>&var</code>	Gets the memory address of <code>var</code>
Dereference	<code>*ptr</code>	Gets/sets the value at the address <code>ptr</code> holds
Pointer declaration	<code>int *p;</code>	<code>p</code> is a pointer to an <code>int</code>
Pointer to pointer	<code>int **pp;</code>	<code>pp</code> points to a pointer to <code>int</code>
Pointer arithmetic	<code>p + 1</code>	Moves forward by one full <code>sizeof(type)</code>
Null pointer	<code>int *p = NULL;</code>	Points to nothing — always check before using
Array & pointer link	<code>arr[i] == *(arr+i)</code>	Array name behaves like a constant pointer

Key takeaway: A pointer is simply a variable that stores an address. Once you can picture memory as a row of numbered boxes, every pointer operation — `&`, `*`, arithmetic, arrays, strings, and double pointers — becomes a matter of following arrows from one box to another.